

Project Title	Blue-Cloud 2026: A federated European FAIR and Open Research Ecosystem for oceans, seas, coastal and inland waters
Project Acronym	Blue-Cloud 2026
Project Number	101094227
Type of project	RIA – Research and Innovation Action
Topics	HORIZON-INFRA-2022-EOSC-01
Starting date of Project	01 January 2023
Duration of the project	42 months
Website	www.blue-cloud.org

Eutrophication Workbench Virtual Laboratory Users handbook

Work Package	WP3 Developing and testing analytical Blue Cloud workbenches for generating highly qualified data collections (Essential Ocean Variables, EOVS)
Task	T3.3 EOVS workbench for eutrophication: chlorophyll, nutrients, oxygen
Lead author	Catalina Reyes (OGS, https://orcid.org/0000-0002-3906-471X)
Contributors	Virginie Racapé (POKaPOK, https://orcid.org/0000-0003-0239-5125) Alban sizun (POKaPOK, https://orcid.org/0009-0006-9387-3709) Gwenaëlle Moncoiffé (NOC-BODC, https://orcid.org/0000-0001-6559-4178) Julie Gatti (Ifremer, https://orcid.org/0000-0001-9327-4366) Maxence Couppey (Ifremer, https://orcid.org/0009-0005-3985-3400) Karin Wesslander (SMHI, https://orcid.org/0009-0005-0981-2421) Alessandra Giorgetti (OGS, https://orcid.org/0000-0002-0914-4831) Megan Anne French (OGS, https://orcid.org/0000-0002-5837-1360) Lotta Fyrberg (SMHI) Robin Kooyman (MARIS) Tjerk Krijger (MARIS) Marine Vernet (Ifremer)
Peer reviewers	Dick Schaap (MARIS), Massimiliano Assante (CNR)
Version	V1.5
Due Date	30/06/2026
Submission Date	30/06/2026

Dissemination Level

x	PU: Public
	CO: Confidential, only for members of the consortium (including the Commission)
	EU-RES. Classified Information: RESTREINT UE (Commission Decision 2005/444/EC)
	EU-CON. Classified Information: CONFIDENTIEL UE (Commission Decision 2005/444/EC)
	EU-SEC. Classified Information: SECRET UE (Commission Decision 2005/444/EC)

Version History

Revision	Date	Editors	Comments
0.1	23/03/2026	Catalina Reyes (OGS)	Updated v1.0
0.2	11/06/2026	all partners	Updated v1.2
0.3	13/06/2026	Catalina Reyes (OGS)	move comments from the master file to the final file. converted to google docs to add codeblock and zotero for references.
0.4	15/06/2026	Gwen Moncoiffe (BODC)	Extended the section: harmonization and standardization.
0.5	19/06/2026	Julie Gatti, Virginie Recape	Comments to the recommendations section.
0.6	25/06/2026	Dick Schaap (MARIS), Massimiliano Assante (CNR)	Review
1.5	29/06/2026	Catalina Reyes (OGS)	Final version for submission

Glossary of terms

Item	Description
BDI	Blue Data Infrastructures
EOV	Essential Ocean Variables
EWB	Eutrophication workbench
NVS	NERC Vocabulary Server
VLab	Virtual Laboratory

Keywords

EOSC; VLabs; Big Data; Virtual Research Environment; Data infrastructures

Disclaimer

The Blue-Cloud 2026 project has received funding from the European Union’s Horizon Europe research and innovation programme under grant agreement No 101094227. Views and opinions expressed are those of the author(s) only and do not necessarily reflect those of the European Union or the European Commission. Neither the European Union nor the European Commission can be held responsible for them.

TABLE OF CONTENTS

1. Introduction.....	6
2. Scope and objectives.....	6
3. Blue Data Infrastructures and Essential Ocean Variables	6
4. Metadata standards and controlled vocabularies.....	7
5. Accessing the workbench	8
5.1. Entry points	8
5.2. Authentication and authorization	8
5.2.1. Authorization and VRE membership (access control)	9
5.2.2. API Tokens for external and notebook access	10
6. System architecture and deployment.....	10
6.1. Beacon data lake	11
6.1.1. Overview	11
6.1.2. Eutrophication workbench (EWB) beacon instances.....	12
6.1.2.1. Monolithic (infrastructure-specific) beacon instances.....	12
6.1.2.2. Federated (merged) beacon instances.....	13
6.1.3. Accessing beacon via a Jupyter Notebook.....	14
6.1.3.1. SDK core resources and repositories.....	14
6.1.3.2. Step-by-step implementation guide	14
6.1.3.3. Practical python code blueprint	15
6.1.4. Harmonization and standardization	16
6.1.4.1. Parameters	17
6.1.4.2. Units	17
6.1.4.3. Platforms	18
6.1.4.4. Instruments	18
6.2. CW Duplicates Tool - CWdt	19
6.2.1. Overview	19
6.3. FileForge - FF	21
6.3.1. Overview and core architecture	21
6.3.2. Supported interoperable output formats.....	22
6.3.3. Upstream pipeline integration and FAIR provenance	22
6.3.4. Running FileForge (FF) on the Cloud Computing Platform (CCP)	22
6.3.4.1. Resource allocations and profile selection.....	22
6.3.4.2. Technical parameter reference guide	23
6.3.4.3. Output management and compression protocols	27

6.3.5. System registries, code access, and documentation	28
6.4. webODV	28
7. Eutrophication workbench workflows.....	29
7.1. EWB VLab directory reference map	29
7.1.1. Master notebook	30
7.2. Core operational principles	30
7.2.1. “Standard” End-User workflow execution.....	31
7.2.1.1. Step 0 - Master notebook.....	32
7.2.1.2. Step 1 - Beacon query	33
7.2.1.3. Step 2 - CWdt.....	33
7.2.1.4. Step 3 - FF	33
7.2.1.5. Step 4 - webODV	34
8. Recommendations for BDIs	34
8.1. Cross-BDI Structural Recommendations	35
8.1.1. Publication and governance of machine-readable vocabularies.....	35
8.1.2. Enrichment of 3 essential metadata : Cruise identifier, Platform and Instrument codes...	35
8.1.3. Implementation of globally unique granular identifiers.....	35
8.2. BDI-Specific Targeted Recommendations	36
8.2.1. EMODnet Chemistry: Adopting standard feature types.....	36
8.2.2. Copernicus Marine Service (CMEMS): Disentangling feature geometries and QC protocols	36
8.2.3. World Ocean Database (WOD): Harmonizing documentation and controlled terms	37
8.3. WOD Controlled Vocabulary Standardization Matrix	37
8.4. Conclusion: Closing the Feedback Loop	38
9. References	39
10. Annexes	39
10.1. Global harmonized and standardized EOv dataset	39

TABLE OF FIGURES

[Figure 1 – Blue cloud gateway sign in.](#)

[Figure 2 – API token access for the beacon D4Science instance.](#)

[Figure 3 – Conceptual architectural workflow and data progression within the Eutrophication Workbench.](#)

[Figure 4 – decision tree.](#)

[Figure 5 – FileForge overview](#)

[Figure 6 – FileForge as CCP method under “Converting Tool” tab.](#)

[Figure 7 – FileForge method execution.](#)

[Figure 8 – Detailed end-user workflow schema.](#)

[Figure 10 – Record counts and time availability of feature types from the Global Harmonized and Standardized eutrophication data collection.](#)

[Figure 11 – Record counts and time availability of WOD dataset type from the Global Harmonized and Standardized eutrophication data collection.](#)

[Figure 12 – Record counts and time availability of CMEMS bigrams from the Global Harmonized and Standardized eutrophication data collection.](#)

[Figure 13 – Record counts vs feature type on each BDI from the Global Harmonized and Standardized eutrophication data collection.](#)

[Figure 14 – Percentage of SeaDataNet Platform categories L06 per SOURCE BDI from the Global Harmonized and Standardized EOVS data collection.](#)

TABLE OF TABLES

[Table 1 – EWB VLab entry points](#)

[Table 2 – Overview of deployed monolithic Beacon instances](#)

[Table 3 – Overview EWB merged Beacon instance.](#)

[Table 4 – FF method execution step by step guide](#)

[Table 5 - Integrated tools matrix](#)

[Table 6 – WOD Vocabulary standardization matrix](#)

[Table 7 – Examples of text found in WOD that would need to be constrained using controlled vocabularies to facilitate interoperability.](#)

[Table 8 – Data availability per BDI from the Global Harmonized and Standardized eutrophication data collection.](#)

1. Introduction

The Eutrophication Workbench Virtual Laboratory (EWB VLab) is a cloud-based environment developed in the D4Science e-infrastructure within the Blue-Cloud 2026 project that enables access to and harmonized use of Essential Ocean Variables (EOVs) relevant to marine eutrophication. It integrates in-situ data from major Blue Data Infrastructures (BDIs) such as the Copernicus Marine Service, EMODnet Chemistry, and the World Ocean Database.

This User's Manual provides a technical description of the workbench, its services, data access procedures, and recommended workflows. It is addressed to scientists, data managers, and policy makers and advisors who wish to merge, validate, and analyze EOVS collections for eutrophication assessment.

2. Scope and objectives

The Eutrophication Workbench aims to:

- Provide a common and customisable, cloud-based workflow for merging, aggregating, and harmonizing multi-source datasets managed by different BDIs.
- Produce validated, customizable EOVS collections for chlorophyll-a, nutrients (nitrate, nitrite+nitrate, ammonium, phosphate, silicate), and dissolved oxygen based on the Global Ocean Observing System (GOOS) recommendations.
- Enable transparent, reproducible data preparation suitable for indicator derivation and regional eutrophication assessments.

The workbench is not a modelling or forecasting environment; it focuses on in-situ data harmonization and standardization, quality control, and exploratory analysis.

3. Blue Data Infrastructures and Essential Ocean Variables

To ensure global scientific relevance, data interoperability, and methodological rigor, the Eutrophication Workbench aligns its data architecture with the internationally recognized framework established by the Global Ocean Observing System (GOOS). Following the guidelines detailed in "GOOS Essential Ocean Variables: the backbone of a sustained and evolving global ocean observing system" (Martín Míguez et al., 2026), the workbench prioritizes a specific subset of Biogeochemical EOVS that are critical for identifying, monitoring, and analyzing marine eutrophication.

The core EOVS accessible through the Eutrophication Workbench include:

- **Ocean color / Phytoplankton biomass (Proxy: Chlorophyll-a):** Used to monitor primary production, algal blooms, and ecological shifts resulting from nutrient enrichment.
- **Oxygen:** Specifically dissolved oxygen, essential for assessing water column ventilation, tracking oxygen depletion, and identifying hypoxic or anoxic "dead" zones.
- **Nutrients:** The foundational drivers of eutrophication. The workbench aggregates and harmonizes data for the following specific macro-nutrients: Nitrate (NO₃), Nitrite + Nitrate (NO₂ + NO₃), Ammonium (NH₄), Phosphate (PO₄) and Silicate (SiO₄).

The EWB acts as a federated data lake engine, harvesting and synthesizing raw datasets from three primary **Blue Data Infrastructures (BDIs)**:

1. **Copernicus Marine Service (CMEMS):** Utilizing the *in situ* Component thematic assembly center to acquire delayed-mode, highly qualified, historically aggregated, and regionally validated marine datasets for operational oceanography data across European seas.
2. **EMODnet Chemistry:** Providing highly qualified, historically aggregated, and regionally validated marine datasets from European regional seas.
3. **World Ocean Database (WOD):** Maintained by NOAA, serving as the world's largest publicly available archive of global ocean data to provide broader geographic coverage.

4. Metadata standards and controlled vocabularies

The core challenge of multi-source integration is data heterogeneity. To eliminate ambiguity when combining assets from CMEMS, EMODnet, and WOD, the workbench enforces strict data syntax and semantic interoperability rules. Metadata standards and controlled vocabularies are dynamically mapped during the ingestion phase. This ensures that regardless of an asset's origin, variables are unified under a singular, standardized vocabulary, allowing users to safely perform cross-infrastructure comparative analyses.

This is based on a list of essential metadata which was defined by experts and that subsequently were mapped to obtain the merged dataset.

5. Accessing the workbench

To interact with the data collections, analytical notebooks, and processing engines, users must first access the EWB VLab environment upon registration on the D4Science e-infrastructure. The workbench is embedded within the broader European open science ecosystem and can be reached via three distinct entry paths, depending on whether you are executing workflows, exploring the service catalog, or accessing via federated European infrastructure.

5.1. Entry points

	Purpose	Primary URL / Registry
Blue-Cloud Gateway - Eutrophication workbench	Interactive execution of tools, workspace access, and notebook environment.	https://blue-cloud.d4science.org/web/eutrophication-workbench
Blue-Cloud Catalogue	Service discovery, metadata inspection, and social sharing of the VLab asset.	https://blue-cloud.d4science.org/web/eutrophication-workbench/catalogue

Table 1 – EWB VLab entry points

5.2. Authentication and authorization

Access to the Eutrophication Workbench Virtual Laboratory is secured via a centralized Identity and Access Management (IAM) framework managed by D4Science. This ensures data security, fine-grained access control and auditability of user interactions with the platform.

Users can authenticate using the D4Science Single Sign-On (SSO) service, which supports federated identity providers. You do not need to create a unique credential set if you belong to a participating institution.

Supported authentication paths include:

- **Institutional Credentials (eduGAIN):** Allows researchers to log in using their primary university or research center credentials via the global eduGAIN federation.
- **Social & Developer Identities:** Supports authentication via authenticated third-party accounts, including ORCID, GitHub, and Google.
- **Native Blue-Cloud/D4Science Accounts:** Users without federated institutional access can register for a direct, native account on the D4Science platform.

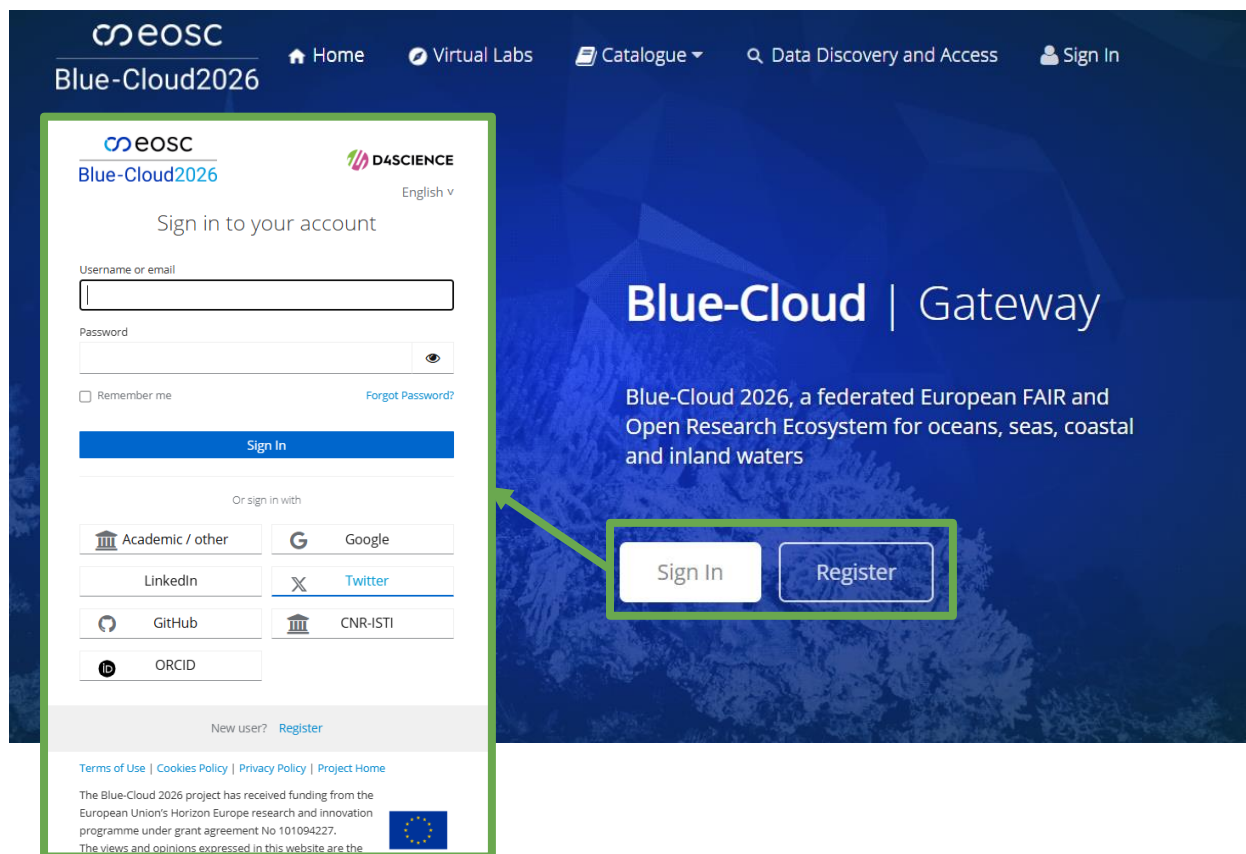


Figure 1 – Blue cloud gateway sign in.

5.2.1. Authorization and VRE membership (access control)

Authentication verifies who you are, while authorization determines what you can do. The EWB operates an open but moderated Virtual Research Environment (VRE) to manage computing resources effectively.

Upon first login, users must **request access** to the EWB VLab via the gateway interface. Once the VRE administrators approve the request, the user account is assigned to a VRE group either Vlab Developers or Vlab End Users.

Once fully authorized, users are granted the following capabilities within the environment:

- **Execute analytical notebooks:** Full permissions to spawn personal JupyterHub instances pre-configured with the computational environment and the beacon-py SDK required to query the data lake.
- **Run cloud computing methods:** Access to the D4Science **CCP** execution tab to configure and launch the CW Duplicates Tool and FileForge engine.

- **Workspace storage and collaboration:** Secure allocation of cloud storage within the VRE Workspace. Users can save filtered Parquet subsets, write intermediate duplicate data reports, and directly share data assets securely with other members of the consortium.

5.2.2. API Tokens for external and notebook access

For programmatic access to the Beacon data lake engine via external Python scripts or internal Jupyter Notebooks, standard browser authentication is insufficient. Users must generate a **Personal Access Token** (<https://blue-cloud.d4science.org/group/eutrophication-workbench/authorization-how-to>) from the EWB VLab dashboard (Figure 3). This token must be passed securely into your initialization scripts (e.g., `client = Client(BEACON_INSTANCE_URL, token=YOUR_TOKEN)`) to authenticate data extraction queries or to explore the swagger service (the latter only available for the maris endpoint).

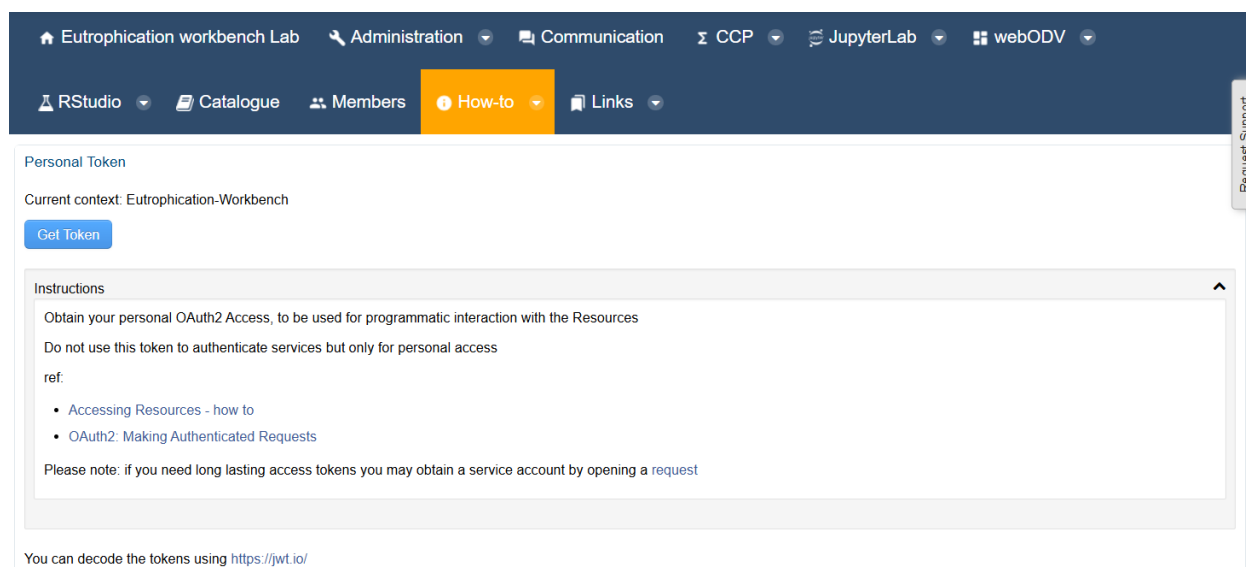


Figure 2 – API token access for the beacon D4Science instance.

6. System architecture and deployment

The EWB VLab is built upon a modular, distributed system architecture designed to handle large-scale marine biogeochemical data. Rather than operating as a rigid, monolithic system, the workbench implements an end-to-end processing workflow (Figure 3) consisting of four core components: **Beacon**, the **CW Duplicates Tool**, **FileForge**, and **webODV**.

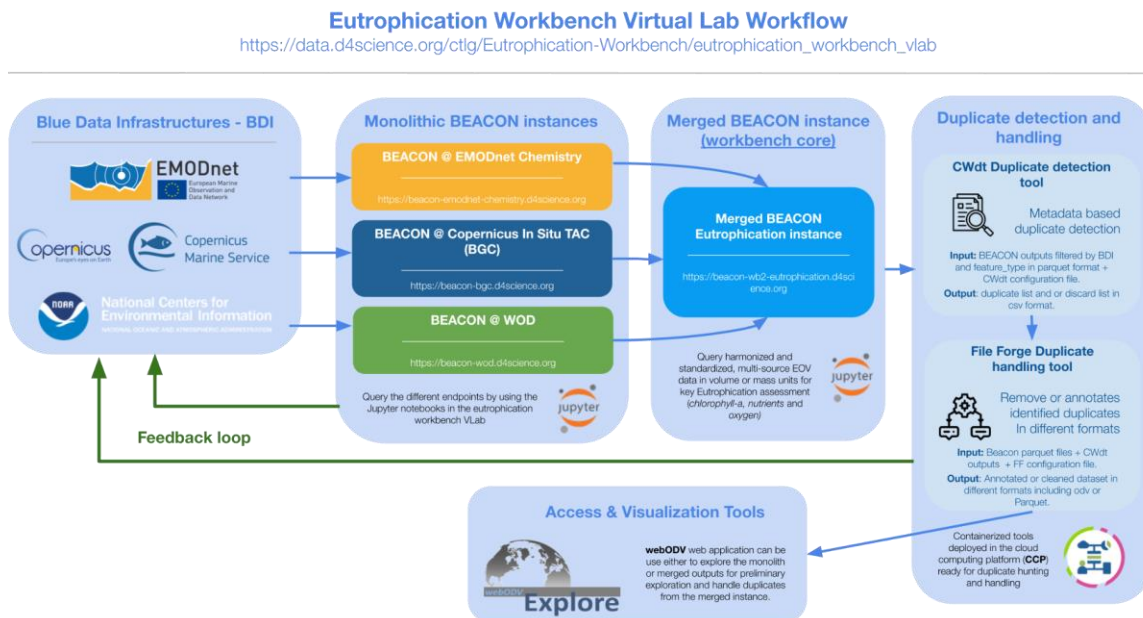


Figure 3 – Conceptual architectural workflow and data progression within the Eutrophication Workbench.

6.1. Beacon data lake

6.1.1. Overview

Beacon is an open-source data lake query engine designed to make large collections of scientific data files directly, easily, and quickly queryable. It focuses primarily on environmental and observational datasets such as oceanographic and climate data but is extendible and proven to work well for other domains as well, as long as the input data follows any standard-like way of describing its contents. Users can request subsets by filtering variables, time ranges, spatial regions, or metadata attributes, and Beacon automatically retrieves and harmonizes the relevant data across all matching files.

In January 2025, Beacon 1.0.0 was made publicly available as an open-source software, allowing everyone to set-up their own Beacon node to enhance the access to their data or use existing Beacon nodes from well-known data infrastructures such as Euro-Argo or the World Ocean Database (WOD) for fast and easy access to harmonized data subsets. More technical details, example applications, GitHub code, and general information on Beacon can be found on the website (<https://beacon-datalake.org/>).

Within the EWB workflow, Beacon acts as the data ingestion and harmonization layer. Instead of requiring users to manually download and cross-reference big data archives from various Blue Data Infrastructures (BDIs), Beacon allows users to request highly specific subsets.

Users can apply dynamic filters based on:

- Specific Essential Ocean Variables (EOVs)
- Temporal ranges (dates)
- Spatial regions (bounding boxes or geographical polygons)
- Specific metadata attributes

Beacon automatically executes these queries across the data lake, retrieves the relevant files, and harmonizes data and metadata schemas into a unified, high-performance structured format in real time.

6.1.2. Eutrophication workbench (EWB) beacon instances

To facilitate flexible, high-performance data exploration, the Eutrophication Workbench deploys Beacon instances across two structural tiers: Infrastructure-Specific (**Monolithic**) Nodes and Federated (**Merged**) Nodes.

These nodes are natively hosted and cross-mirrored between MARIS and the D4Science Cloud Platform, providing data redundancy and high availability. Users can interact with these endpoints directly through pre-configured Jupyter Notebooks within the EWB VLab.

6.1.2.1. Monolithic (infrastructure-specific) beacon instances

Monolithic instances isolate individual Blue Data Infrastructures (BDIs) into distinct, dedicated query engines. These nodes are optimized for targeted queries where a researcher needs to inspect, evaluate, or extract data from a single specific institutional authority without cross-contamination from other networks.

Using monolithic instances is highly recommended during the initial exploration phases, for validating data provenance, or when performing comparative studies between different source infrastructures.

Blue Data Infrastructure	Source Dataset & Collection Criteria	Beacon MARIS Endpoint	Beacon D4Science Endpoint	Blue cloud/EOSCnodeDTO catalogue entry
World Ocean Database (WOD)	Actual depth profiles, harvested directly from NCEI/NOAA (ncei.noaa.gov).	beacon-wod.maris.nl	beacon-wod.d4science.org	https://data.d4science.org/ctlg/EOSCNodeDTO-Catalogue/beacon-world_ocean_database
EMODnet Chemistry	Highly qualified EMODnet	beacon-emodnet-chem.maris.nl	beacon-emodnet-chem.d4science.org	https://data.d4science.org/ctlg/EOSCNodeDTO-EMODnet_chemistry

	Chemistry Collection (v2025), retrieved from WebODV.		chemistry.d4science.org	Catalogue/beacon - emodnet chemistry
Copernicus Marine Service (CMEMS)	Biogeochemical (BGC) discrete in situ datasets: INSITU_GLO_BGC_DISCRETE_MY_013_046	beacon-cmems.maris.nl	bgc.d4science.org	https://data.d4science.org/ctlg/EOSCNodeDTO-Catalogue/beacon - cmems_bgc

Table 2 – Overview of deployed monolithic Beacon instances

6.1.2.2. Federated (merged) beacon instances

Going beyond individual repository isolation, the Eutrophication Workbench demonstrates true cloud-native data federation by combining the data pipelines of all three monolithic sources into a unified, cross-infrastructure network layer.

The **Merged beacon instance** aggregates the contents of the WOD, EMODnet Chemistry, and CMEMS BGC repositories into a singular queryable node. Instead of writing multiple query blocks and manually appending separate datasets, a user can broadcast a single spatial-temporal query to the merged node. The engine processes the request globally across all three underlying collections and outputs a unified dataset in the desired format, for example Apache Parquet (see the supported formats section at: <https://maris-development.github.io/beacon/>).

Combined Federated Collections	Beacon MARIS Endpoint	Beacon D4Science Endpoint	Blue cloud/EOSCnodeDTO catalogue entry
EMODnet Chemistry, WOD, CMEMS BGC	beacon-wb2-eutrophication.maris.nl	beacon-wb2-eutrophication.d4science.org	https://data.d4science.org/ctlg/EOSCNodeDTO-Catalogue/beacon - merged_eutrophication

Table 3 – Overview EWB merged Beacon instance.

⚠ Important data processing note: Because the merged beacon instance represents an absolute aggregation of overlapping data infrastructures, the raw output datasets will contain spatial and temporal observation duplicates. This merged dataset is designed to serve as the direct upstream input file for the CW Duplicates Tool (see Section 4.2), which identifies duplicate artifacts before analysis.

6.1.3. Accessing beacon via a Jupyter Notebook

Programmatic interaction with any deployed Beacon endpoint (detailed in Section 6.1.2) is driven by the **Beacon API**. Rather than forcing researchers to write raw HTTP REST requests manually, the EWB VLab leverages a native, purpose-built library: the Beacon Python Software Development Kit (SDK).

6.1.3.1. SDK core resources and repositories

- Production source code and library: <https://github.com/maris-development/beacon-py>
- Official EWB template and workflow Notebooks: https://github.com/maris-development/beacon-blue-cloud/tree/EutrophicationWorkbench_CR/notebook-examples/EutrophicationWorkbenchVLab

6.1.3.2. Step-by-step implementation guide

To construct a notebook workflow that extracts harmonized subsets directly from the data lake, users follow a structured four-stage execution pipeline:

1. **Environmental setup and client initialization:** First, ensure the environment has the SDK installed (`pip install beacon_api`). In your notebook, import the client modules and establish an authenticated connection to the targeted Beacon endpoint (e.g., the World Ocean Database node). You must pass your personal D4Science Access Token to clear the security layer.
2. **Defining multi-dimensional query criteria:** Queries to the data lake engine are highly targeted to prevent memory bottlenecks. The SDK exposes an abstraction layer to filter data collections across four primary dimensions:
 - a. **Variables:** Specifying target parameters using vocabulary-controlled keys depending on the queried beacon instance (e.g., `COMMON_OXYGEN` in the merged or `Water body nitrate` in the EMODnet chemistry monolithic instance).
 - b. **Spatial bounding box:** Constraining the search space using geographic coordinates (`min_lon`, `max_lon`, `min_lat`, `max_lat`).
 - c. **Temporal interval:** Defining the exact chronological boundaries (`start_date`, `end_date`).
 - d. **Metadata property matrices:** Narrowing the scope further based on infrastructure-specific criteria like asset platform type (e.g., *moored buoy*, *towed CTD*) or data originating research institutes (e.g., *EDMO code*).
3. **Query submission and data stream assimilation:** Once compiled, the query object is transmitted via the SDK to the remote Beacon engine. The engine transparently parses the target (e.g. Apache Parquet) files across the distributed data lake, applies your masks, aggregates the matching indices, and prepares the downstream delivery bundle.

To maximize the transparency and reproducible nature of marine data integration within the Eutrophication Workbench, the reference implementation for querying the EMODnet Chemistry monolithic instance is published openly as a production-ready Jupyter Notebook.

6.1.3.3. *Practical python code blueprint*

Below is the concrete, production-grade Python implementation extracted from the official [EMODnet Chemistry monolithic BEACON query notebook](#) (Reyes Suarez, 2026a). This blueprint establishes a client connection, configures explicit column selections with interoperable aliases, applies strict spatiotemporal and quality filters, and fetches the data directly into an analytical structure.

```
# =====
# BLUE-CLOUD 2026: EUTROPHICATION WORKBENCH VIRTUAL LABORATORY
# Practical Python Code Blueprint: Querying EMODnet Chemistry via Monolithic BEACON
# =====

from beacon_api import * # Import the Beacon API client

TOKEN = os.getenv('D4SCIENCE_TOKEN') # This will fetch the token from the VRE environment.
print("Initializing connection to the EMODnet Chemistry monolithic Beacon node...")

# Define the target Monolithic Beacon Endpoint URI
BEACON_INSTANCE_URL = "https://beacon-emodnet-chemistry.d4science.org"

# Instantiate the Beacon Client
emodnet_client = Client('https://beacon-emodnet-chemistry.d4science.org', jwt_token=TOKEN)

print(f"Successfully connected to endpoint: {BEACON_INSTANCE_URL}")

# Initialize the Step-by-Step Query Builder
query_builder = client.query()

print("\nConfiguring structural metadata and tracking columns...")
# Select core identifier and institutional metadata columns
query_builder.add_select_column("Originator")
query_builder.add_select_column("LOCAL_CDI_ID")
query_builder.add_select_column("EDMO_code")

# Select primary spatial vertical context dimensions
query_builder.add_select_column("yyyy-mm-ddThh:mm:ss.sss", alias="time")
query_builder.add_select_column("Latitude", alias="latitude")
query_builder.add_select_column("Longitude", alias="longitude")
query_builder.add_select_column("Depth", alias="depth")

# -----
# PARAMETER SELECTION EXAMPLES
```



```
# -----
query_builder.add_select_column("Water body chlorophyll-a", alias="Water body chlorophyll-a")
query_builder.add_select_column("Water body chlorophyll-a_qc", alias="Water body chlorophyll-a_qc")

print("\nApplying spatiotemporal constraints and quality filters...")
# Define bounding criteria (Modify constraints based on regional focus)
query_builder.add_range_filter("TIME", "2010-01-01T00:00:00", "2024-12-31T23:59:59") #
Temporal range
query_builder.add_range_filter("LATITUDE", 30.0, 46.0) #
Mediterranean example bounding box
query_builder.add_range_filter("LONGITUDE", -6.0, 36.0) #
Mediterranean example bounding box
query_builder.add_range_filter("DEPTH", 0.0, 250.0) #
Euphotic/epipelagic zone limit

# Execute query to fetch the harmonized dataset from the data lake
print("Submitting query and executing remote server aggregation...")
df = query_builder.to_pandas_dataframe()

print(f"\nData extraction completed successfully!")

print("\nFirst 5 rows of the FAIR-harmonized DataFrame:")
print(df.head())
# Download in parquet format
query_builder.to_parquet()
```

6.1.4. Harmonization and standardization

The harmonisation and standardisation effort within Blue-Cloud2026 aims at enabling faster integration, more accurate duplicate checks, and greater interoperability while minimising information loss and maintaining traceability.

It was applied to the diverse unit, coordinate reference system, time format and quality flag conventions used across the source BDIs, as well as the different labelling conventions used for variable names, observation platforms, sensors and sampling devices.

The general approach to standardisation and semantic harmonisation within the Blue-Cloud2026 project is described in a number of Deliverables (e.g. [D3.1](#) and [D5.5](#)). In this section, we will focus on information related to the practical implementation of semantic harmonisation for the EWB.

The strategy and rules followed for the semantic harmonisation are as follows:

6.1.4.1. Parameters

The BODC P01 Parameter Usage Vocabulary (<http://vocab.nerc.ac.uk/collection/P01/>) was the only parameter naming vocabulary capable of accommodating the range of variable naming practices in use in the different source BDIs. Using either SKOS mappings stored in the NVS, or mappings hardcoded in Beacon, source parameter labels were matched to their exact or closest broader P01 parameter concept based on knowledge of what these labels represented.

CMEMS BGC: the P09 MEDATLAS Parameter Usage Vocabulary (<http://vocab.nerc.ac.uk/collection/P09/>) used by CMEMS for parameter labelling are already matched to their respective P01 concepts by the P09 governance team at Ifremer hence these mappings were already available from the NVS.

EMODnet Chemistry: the P35 EMODnet Chemistry aggregated parameter names (<http://vocab.nerc.ac.uk/collection/P35/>) used by EMODnet Chemistry for their aggregated data products were matched to their equivalent P01 concept. These one-to-one mappings are different from the ones already present in the NVS. The latter are needed for the dynamic harvesting of data from SeaDataNet to EMODnet Chemistry. In order to avoid the risk of disrupting the existing workflow, the Blue-Cloud mappings were hardcoded in Beacon.

WOD: uses its own internal codelists (published in the WOD Manual and web pages). These were mapped manually to P01 codes and stored in Beacon.

6.1.4.2. Units

Source units and their labels were standardised using the NVS P06 vocabulary (<https://vocab.nerc.ac.uk/collection/P06/current/>).

Source units are read by Beacon and mapped to their matching concept from the NVS P06 vocabulary uniquely identified in Beacon by its URN (e.g. SDN:P06::KGUM or SDN:P06::UPOX) and preferred labels.

6.1.4.3. Platforms

The common vocabulary used for platform harmonisation was the NVS L06 SeaVoX Platform Categories vocabulary (<http://vocab.nerc.ac.uk/collection/L06/>). However duplicate record identification across all BDIs but Argo, also benefitted from having not only the platform type but also the platform instance accurately identified. This was done by mapping all non-Argo platform instances to the ICES platform code vocabulary using their NVS C17 unique URN (e.g.

SDN:C17::74E3). Different methods had to be followed to map the source information from the various BDIs to their correct C17 code. Below is a summary of the different approaches.

CMEMS: values in “ship_platform_number” are matched to the call-sign in C17. Beacon uses the time at which the observation was made to ensure that the call-sign is matched to the correct platform instance in the C17 vocabulary. L06 platform type URNs are derived from the CMEMS Data Type Bigram with a conditional criteria based on the value of the WMO_instrument_type.

EMODnet chemistry: the C17 platform instance and/or L06 platform type are extracted from the CDI metadata record and directly inserted in their appropriate fields in Beacon.

WOD: WOD maintains its own list of codes and names for platform instances¹. The WOD list was extracted and manually mapped to C17 concepts using a series of iterative matching methods that used a succession and combination of criteria including platform names, classes, countries, call-signs, IMO numbers. A total of 470 platform (mainly ship) instances were mapped to their unique ICES platform identifier from C17 (around 50% of the platform instances held in WOD) with 100% certainty. These mappings were used by Beacon to match incoming WMO platform IDs to a C17 URN and, using the mapping held in the NVS between C17 and L06, to the L06 category. These mappings have also been added to the Knowledge Base of the BODC Semantic Analyser as blank node RDF triples so that they are available for re-use.

6.1.4.4. *Instruments*

In Beacon, “instrument” is variable-specific and it represents a key element of the method used to create a measurement. The common vocabulary selected for the harmonisation of instruments (whether a sensor or a sampling device) is the NVS L05 SeaDataNet device categories vocabulary (<http://vocab.nerc.ac.uk/collection/L05/>) which captures instrument types rather than instrument models. Whenever possible, instrument models were also matched to their unique identifier from the NVS L22 SeaVoX Device Catalogue. However this information was not often available or it was not always easy to link it to a given variable. A review of how data models capture those elements of methodology will be needed in the future to ensure that a parameter can be easily connected to the method used to derive it, and that this information is readily usable by software.

- **CMEMS:** the instrument field for each parameter was populated from a mapping between CMEMS WMO_INST_TYPE and L05 or L22, hardcoded in Beacon. Failing this, Beacon uses

¹ https://www.ncei.noaa.gov/access/world-ocean-database/CODES/s_3_platform.html

the CMEMS Data Type Bigram which was manually mapped to L05, to fill the gaps. It was not always possible to populate this field and, in the future, effort will be needed to better harmonise this information across the BDIs.

- **EMODnet Chemistry:** the variable-specific L22 code is obtained from the metadata header of the SeaDataNet data file, information which is kept in the EMODnet Chemistry data product. If this field is empty, then Beacon uses the L05 code held in the CDI metadata record. However this field can contain multiple L05 identifiers if the data come from devices belonging to more than one device category. A review of the relationships between instruments and variables at the CDI discovery levels and/or more consistent use of the L22 field in the data file header would help resolve this issue and improve recording or provenance and traceability.
- **WOD:** instrument type and model information was extracted from WOD code table v_5_instrument² which also contains information about platforms, and mapped to an exact L22 concept or the closest broader L22 or L05 concepts. These mappings are currently hardcoded in Beacon but will be made available via the BODC Semantic Analyser.

6.2. CW Duplicates Tool - CWdt

6.2.1. Overview

Because the EWB harvests data from overlapping global and regional infrastructures (such as CMEMS, EMODnet, and WOD), the raw extracted subsets frequently contain duplicate station observations. The **CW Duplicates Tool** is tasked with identifying and flagging these spatial and temporal duplicates.

Developed by the POKaPOK team (<https://pokapok.org>) the tool evaluates potential duplicates based on mandatory metadata profiles (e.g., exact coordinate proximity, overlapping timestamps, and instrument types) alongside user-defined tolerance criteria.

The tool executes a multi-step analytical process:

1. **Detection and statistics:** Scans the harmonized input datasets to build a statistical matrix of potential duplicates.
2. **Decision tree evaluation:** Applies a programmatic decision tree (figure 5) to assess which duplicate record possesses the higher quality flags, richer metadata, or better vertical resolution.

² https://www.ncei.noaa.gov/access/world-ocean-database/CODES/v_5_instrument.html

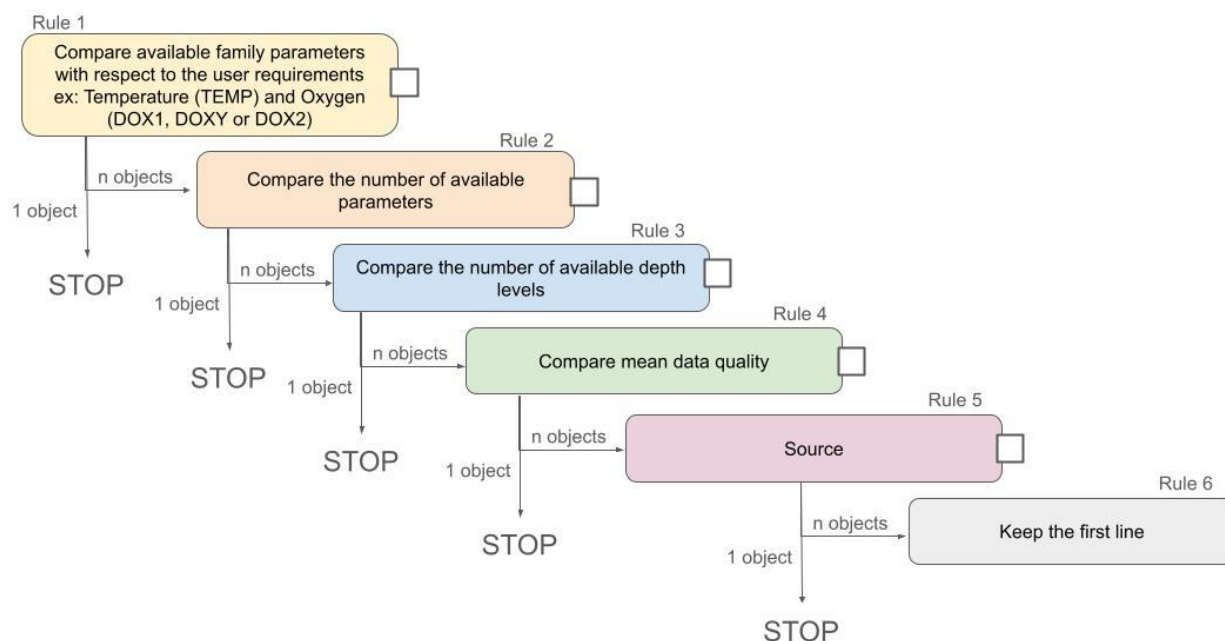


Figure 4 – decision tree.

3. **Output generation:** Produces two primary downstream files in comma-separated values (CSV) format:
 - a. *The Duplicates List:* A comprehensive log mapping all paired redundant records for quality control transparency.
 - b. *The Discard List:* A precise exclusion list specifying which exact rows or profiles must be filtered out to generate a clean, unique dataset product.

Deployment and access: The CW Duplicates Tool is deployed as an executable method on the [D4Science Cloud Computing Platform](https://blue-cloud.org) under the "Duplicates-tools" method tab. The complete source code is open-source and archived on Zenodo (<https://doi.org/10.5281/zenodo.20605786>), with a comprehensive operating guide located within the workbench workspace at Eutrophication-workbench/0-Documentation.

Find the resource in the blue-cloud catalogue under the Eutrophication workbench Vlab: https://data.d4science.org/ctlg/Eutrophication-Workbench/cw_duplicates_tool.

6.3. FileForge - FF

6.3.1. Overview and core architecture

FileForge is a data transformation engine developed by [lfremer](#) that bridges the gap between raw cloud-optimized analytical data and standard desktop oceanographic software formats. It acts as the workflow's compilation layer, consuming the high-volume Parquet datasets produced by Beacon and refining them based on the quality control actions dictated by the CW Duplicates Tool.

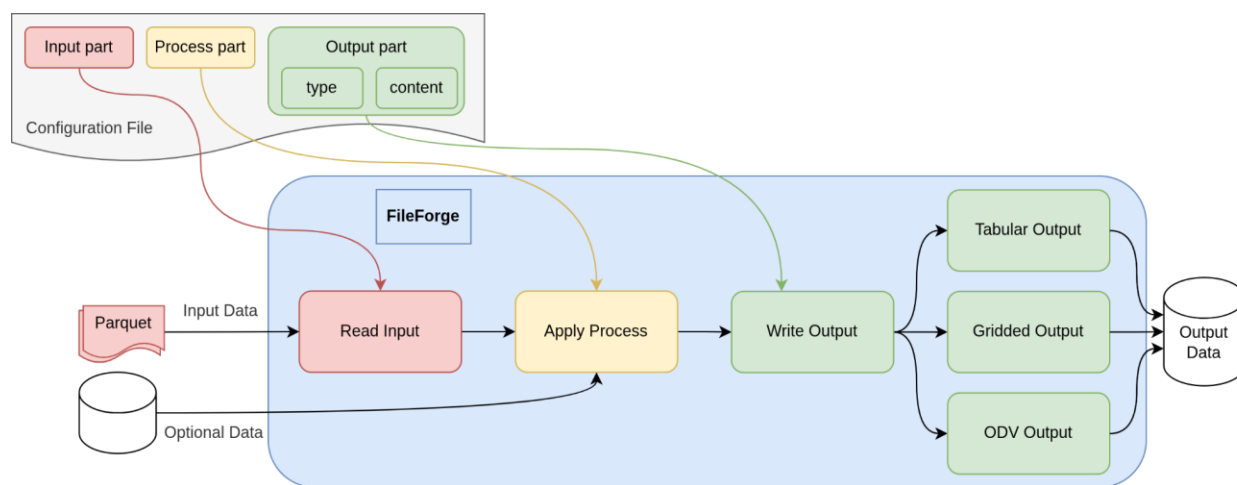


Figure 5 – FileForge overview

Architecturally, the tool executes its pipeline across three distinct modules:

1. **Input ingestion and flattening layer:** Reads multidimensional spatial data models and flattens them into tabular in-memory objects (such as Pandas or Polars DataFrames). If external descriptive metadata arrays are supplied, they are systematically mapped and appended to their corresponding observation rows.
2. **Processing application layer:** Applies user-defined transformation algorithms, QC routines, and filtering matrices to the active dataset using customized configuration arguments.
3. **Format-specific writing layer:** Constructs the final file structure. Because different data specifications handle file headers and global variables differently, this module customizes metadata indexing to exploit the strengths of each format type while strictly preserving structural data integrity.

6.3.2. Supported interoperable output formats

Based on a user-defined configuration execution template, FileForge can reorganize and compile active datasets into three primary computational formats, maximizing downstream system interoperability:

- **Apache Parquet (tabular formats):** High-performance, columnar storage layouts optimized for distributed, cloud-native big data analytics.
- **ODV Spreadsheet collections (tabular station data):** Formatted and column-indexed specifically for native desktop consumption or cloud deployment within the Ocean Data View (ODV) and webODV ecosystems.
- **NetCDF files (gridded and profile formats):** Self-describing, Climate and Forecast (CF) metadata-compliant array files tailored for climate modeling, data assimilation, and spatial Geographic Information Systems (GIS).

6.3.3. Upstream pipeline integration and FAIR provenance

Within the EWB workflow, FileForge functions as the terminal consolidation engine. It directly consumes high-volume, Apache Parquet datasets harvested by the Beacon data lake engine and refines them using the analytical outputs of the CW duplicates Tool.

During this compilation phase, FileForge dynamically parses the `Duplicate List` and `Discarded List` produced during the deduplication phase. The engine filters out duplicate profiles while embedding full tracking flags that trace records back to their original institutional source infrastructures. Simultaneously, FileForge automatically populates processing logs to document exact software versions, runtime parameters, and processing methods. This comprehensive lineage tracking ensures strict compliance with **FAIR data provenance principles**, making the generation of any data product completely transparent and reproducible.

6.3.4. Running FileForge (FF) on the Cloud Computing Platform (CCP)

FileForge is exposed as an executable cloud service on the **D4Science Cloud Computing Platform (CCP)** under the "**Converting Tool**" method group.

6.3.4.1. Resource allocations and profile selection

When launching the tool, users can select between two distinct server profiles* depending on data volumes and current cluster queue states:

- **16 GB RAM Profile:** The standard processing node configuration. This instance operates efficiently for regional workflows (for example, compiling the Mediterranean Sea EOVS collections spanning 1950–2025). Because D4Science maintains a high volume of 16 GB nodes, scheduling queues for this instance are typically short or non-existent.
- **32 GB RAM Profile:** Hosted on advanced, hardware-accelerated GPU cluster nodes. This profile provides superior operational speeds and expanded memory capacity for **global collections**. However, because these nodes are shared across intensive Virtual Research Environments (VREs), users may encounter brief scheduling queues if concurrency is high.

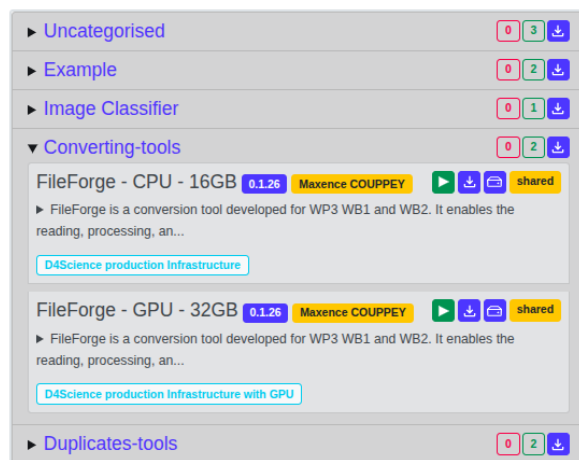


Figure 6 – FileForge as CCP method under “Converting Tool” tab.

* The configuration fields and operating procedures are identical across both the 16 GB and 32 GB profiles.

6.3.4.2. Technical parameter reference guide

To execute a FF conversion routine, open the method interface by clicking the green execution icon on the CCP tab (Figure 7). the method configuration will appear on the right hand side of the window as shown in Figure 8. The step by step guide to configure the execution method and parameter fields needed are explained in Table 4 below.

gitlab-registry.frermer.fr/insea/insea-exporter:0.1.26

*** Runtime size**
The size for a runtime to execute a method. For container based infrastructures: Small allocates up to 1 CPU and 4GB ram, medium (default) allocates up to 2 CPUs and 8G ram, large allocates up to 4 CPUs and 16Gb ram.

large

*** Configuration File**
The configuration file to use

Configuration File

*** Data folder**
Folder that contains input data(s)

Data folder

External Folder
(Optional) External information Folder, which contains some informations you would like to use in FileForge. I.e CloneWars data.

External Folder

*** Logging Level**
Logging level of the output

DEBUG

*** Abort execution on error**

☒

Destination Folder
(Opt.) Destination output Folder. If set, Output data will be written in this folder, else in your default `/workspace/cplexecution/...`. Please consider using method if the output files are big.

Destination Folder

*** Annotation**
The value of this parameter will be associated as annotation to the execution

Annotation

Options

Select what you like to archive automatically when the execution is completed.

Automatically archive whole execution provenance

Outputs

☒ Standard error ☒ Standard output

Execute

Generate code for
Python 3

Figure 7 – FileForge method execution.

Parameter Field	Operational Requirement & Expected Format
Configuration file	The absolute URL or VRE workspace path pointing to the execution template. Users can pass an individual YAML configuration file, or point directly to a parent workspace directory to execute all nested configuration templates sequentially.

	Access your workspace Select an item or drag and drop it to a proper input 2_CWduplicatesTool 3_FileForge Configuration_files wb2_netcdf_config_2.yaml wb2_netcdf_config_2_chunked.yaml wb2_netcdf_config_2_clean.yaml SELECT or Access your workspace Select an item or drag and drop it to a proper input wb2_netcdf_config_2_clean.yaml wb2_odv_config_2.yaml wb2_odv_config_2_chunked.yaml wb2_odv_config_2_clean.yaml wb2_odv_config_timeseries.yaml wb2_parquet_config_2.yaml SELECT
--	---

		Access your workspace Select an item or drag and drop it to a proper input outputs 20260612162118_cw_duplicates_tools_output out cache log SELECT
Destination (optional)	folder	Specifies the cloud storage destination for the generated output products. If left blank, the tool defaults to saving final data objects into the root directory of your personal user Workbench folder. Access your workspace Select an item or drag and drop it to a proper input 2_CWduplicates-tool 3_FileForge Configuration_files output Eutrophication-Workbench SELECT
Annotation (optional)		A customizable string to override the default timestamp folder nomenclature (YYYYMMDDhhmmss_fileforge). For example, defining the annotation string as med_chlorophyll_final forces FileForge to name the output directory med_chlorophyll_final_fileforge. This protects historical datasets from being overwritten and simplifies multi-version comparison tracking. * Annotation The value of this parameter will be associated as annotation to the execution my_first_result

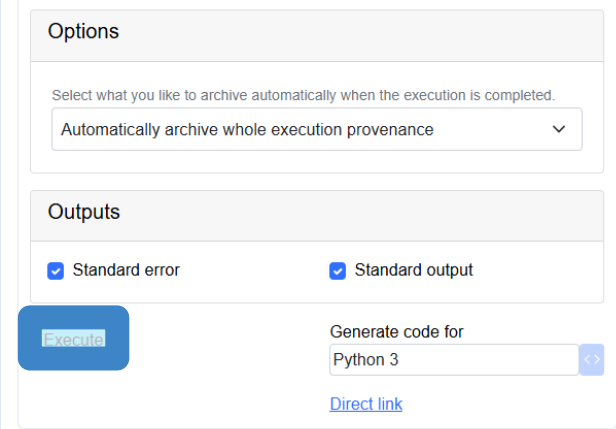
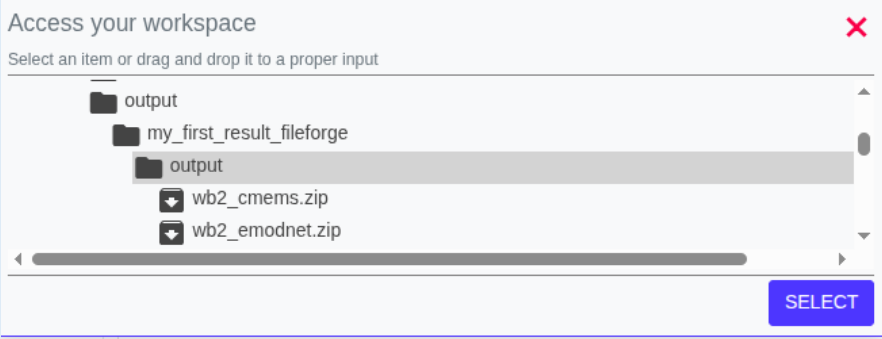
Method execution	After having configured the above, the method can be executed. 
	When the method is successful, the results will be shown in the destination folder. 

Table 4 – FF method execution step by step guide

6.3.4.3. *Output management and compression protocols*

Upon a successful execution run, the logs and finalized data objects are deployed to the assigned destination directory.

When generating webODV Spreadsheet Collections, FileForge automatically bundles the plain-text spreadsheet assets inside compressed .zip archives. Because raw oceanographic ODV text matrixes can expand exponentially, this compression routine drastically simplifies web visualization loading times and file transfers (for instance, compressing an unzipped 80 GB text collection down to an optimized 5 GB archive package).

6.3.5. System registries, code access, and documentation

FileForge is deployed as a cloud service method on the [D4Science Cloud Computing Platform](#). The underlying source code is maintained on Ifremer's GitLab instance available on <https://zenodo.org/records/20596096>.

Find the resource in the blue-cloud catalogue under the Eutrophication workbench Vlab: https://data.d4science.org/ctlg/Eutrophication-Workbench/fileforge_in_eutrophication_workbench_ccp

Comprehensive Operator Technical Manual <https://data.d4science.net/5B1Tt>

6.4. webODV

The final component of the workbench architecture is **webODV**, a web-based implementation of the gold-standard **Ocean Data View (ODV)** software. It serves as the interactive visualization, exploration, and delivery layer of the Eutrophication Workbench ecosystem.

Rather than forcing users to download massive raw files locally to visually inspect their data, webODV hosts the finalized, deduplicated, and FileForge-processed data collections directly in the cloud. It provides an intuitive, browser-based user interface that replicates the core capabilities of the desktop ODV application.

Through webODV, users can:

- **Interactive Visualization:** Instantly generate high-quality station maps, vertical section plots, scatter plots (such as Temperature vs. Salinity or Chlorophyll-a vs. Nutrients), and temporal surface animations.
- **Quality Control Inspection:** Interactively examine individual station profiles, visually identify outliers, and check data distribution across distinct bathymetric depths or marine regions.
- **Custom Subsetting & Extraction:** Apply fine-tuned visual filters directly on the canvas to isolate specific data points, and subsequently export these tailored subsets into regional data packages for localized research.

By integrating webODV as the terminal node of the architecture, the workbench ensures that the final data products are immediately actionable, enabling swift scientific discovery and assessment of marine eutrophication trends without local software dependencies.

Deployment & Access: [webODV](#) has been deployed as a web application in jupyterhub. The software is developed and maintained by Dr. Sebastian Mieruch and Prof. Dr. Reiner Schlitzer at the Alfred Wegener Institute (AWI), and its specialized manuals can be referenced at:

- Getting started with webODV: <https://explore.webodv.awi.de/docs/odv-online-getting-started.pdf>
- webODV How-to guide: <https://explore.webodv.awi.de/docs/odv-online-howto.pdf>
- ODV Full documentation: <https://odv.awi.de/documentation>
- ODV blue cloud deployment documentation: after starting the webapp go to the tab Docs.

7. Eutrophication workbench workflows

Operational workflows within the EWB VLab are split into two primary tracks tailored to the user's objective. For example, if the user wants to reproduce the EWB H&S EOVS data collection output or adapt the workflow to its own needs. The two groups as described in [section 5.2.1](#) are: the **Standard End-User Workflow** (a guided, step-by-step hybrid pipeline) and the **Developer/Advanced User Workflow** (for component customization and direct API and method manipulation).

7.1. EWB VLab directory reference map

To maintain structural clarity, the following layout details the role of each directory within the [EWB VLab workspace](#) ecosystem:

```
VRE Folders/Eutrophication-Workbench/
├── 0_Documentation/ # System and user guides, and tool technical manuals
├── 1_BEACON/ # Core BEACON data-querying notebooks examples.
├── 2_CWduplicates-tool/ # Deduplication YAML configurations, metadata rules, and Logs
├── 3_FileForge/ # Format transmutation layouts and YAML configuration files
├── 4_webODV/ # Synced ODV data spreadsheets and binary collections
├── REPO/ # EWB repository, old test
├── user_workflow/ # Contains the master notebook for the user workflow initialization.
└── README.md # Root environment overview file
```

The [README.md](#) describes the standard EWB workflow and provides descriptions of expected inputs and outputs on each step.

7.1.1. Master notebook

Due to the lack of an orchestration layer, the master notebook in `workspace/VREFolders/Eutrophication-Workbench/user_workflow/0_Start_Here_RUNME.ipynb` and published in the blue cloud catalogue (Reyes Suarez, 2026b) aims to set up the complete folder structure and configuration files needed to run the workflow.

1. Open [JupyterLab](#) in the EWB VLab.
2. Navigate to your home workspace:
`/home/jovyan/workspace/VREFolders/Eutrophication-Workbench/user_workflow/`
3. Create a **copy** of the folder:
`../user_workflow/0_Start_Here_RUNME.ipynb` in your `/home/jovyan/workspace/`
4. Open and run the notebook in your copied folder.
5. When prompted, enter:
 - Your initials (user identifier)
 - Start year and month (YYYYMM format)
 - End year and month (YYYYMM format)
 - Depth range from (metres)
 - Depth range to (metres)

once run, the user will find the following directory tree created:

```
[root]/
├── 1_DatalakeQuery/
│   └── Merged_notebooks/
│       ├── 01_EWB_BEACON=merged-v1.5.4_FILTER=BDI-FeatureType_VAR=allEOV_OUT=parquet.ipynb
│       └── output/
├── 2_CWduplicates-tool/
│   ├── cache_empty/
│   ├── Configuration_files/
│   │   └── Config_Profiles/
│   │       └── cw_user_config.yaml
│   └── outputs/
└── 3_FileForge/
    ├── Configuration_files/
    │   └── [All files copied from the source FileForge config folder]
    └── output/
```

7.2. Core operational principles

Regardless of the workflow track, the entire infrastructure operates under strict scientific constraints to guarantee transparency, reproducibility, and rigorous data provenance:

- **Hybrid execution framework:** The EWB workflow is intrinsically hybrid; it is not fully automated and intentionally relies on user checkpoint decisions and manual interventions to maintain scientific control.
- **Isolated environments:** Each analytical step runs inside its own dedicated execution environment (JupyterLab, CCP, or webODV), preventing environment pollution and cross-tool dependency conflicts.
- **Core EOVS Coverage:** The workflows are optimized to ingest, clean, and export data for the EOVSs described in [section 3](#).

The functional steps across both workflows depend on the four core system components ([section 6](#)) integrated into the EWB VLab. Keep in mind (as stated in [section 5](#)) that to interact with the data collections, analytical notebooks, and processing engines, users must first access the EWB VLab environment upon registration on the D4Science e-infrastructure:

Component	Purpose	Where to Access
Beacon	Data access, subsetting, merging, harmonization	JupyterLab tab using beacon endpoints
CWdt	Detect and classify duplicates	Inside the CCP tab
FF	Remove annotated duplicates, convert to ODV	Inside the CCP tab
webODV	Explore and extract ready-to-use datasets	WebODV tab

Table 5 - Integrated tools matrix

7.2.1. “Standard” End-User workflow execution

The end-user workflow leverages **pre-packaged scripts** and **configuration templates** to systematically guide researchers from raw BDI to an analysis-ready oceanographic collection ([Figure 3](#)). These steps are summarized in the [EWB welcome page](#) as well.

As described above if the objective is to reproduce or adapt the EWB workflow the user must follow the steps below and detailed in figure 9.

[Step 0: Setup] → [Step 1: BEACON Query] → [Step 2: CW Duplicates] → [Step 3: FileForge] → [Step 4: webODV]

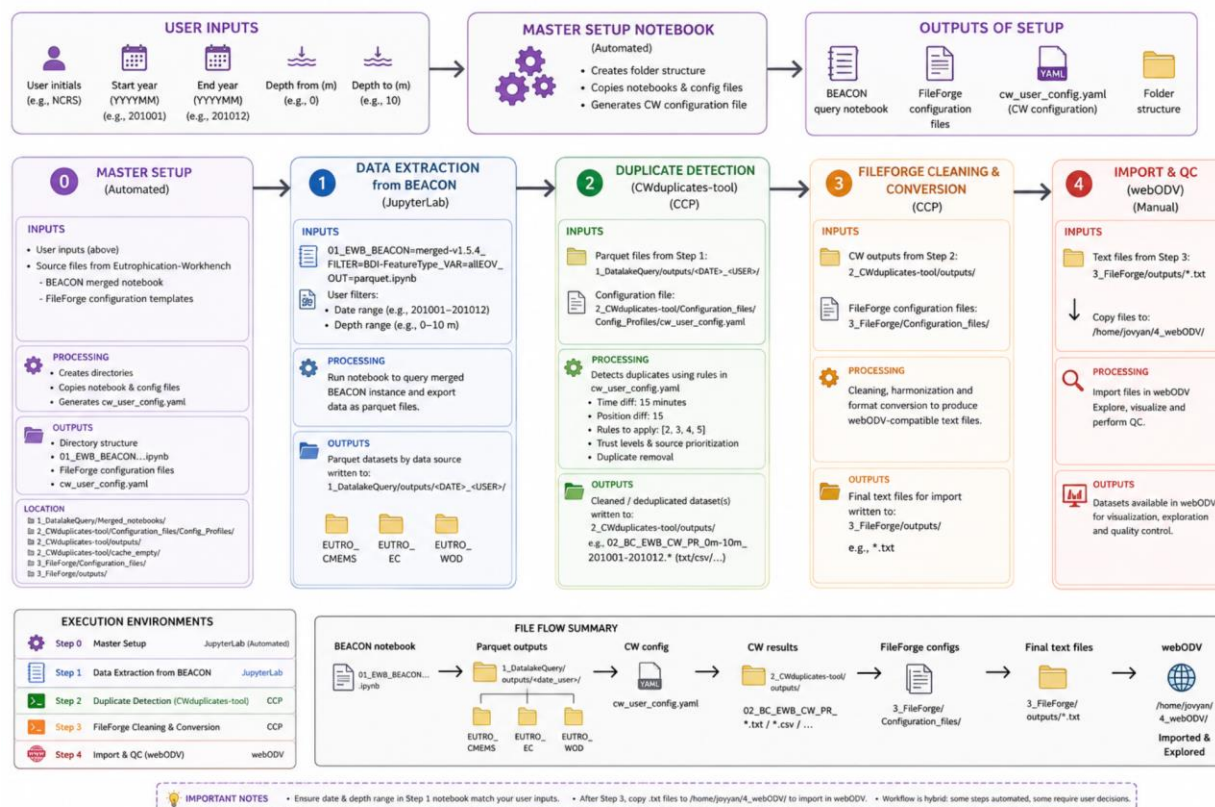


Figure 8 – Detailed end-user workflow schema.

7.2.1.1. Step 0 - Master notebook

- Launch JupyterLab from the top menu of the EWB VLab dashboard.
- Locate the master template folder:
/home/jovyan/workspace/VREFolders/Eutrophication-Workbench/user_workflow/
- Copy the setup notebook 0_Start_Here_RUNME.ipynb directly into your root home workspace directory (/home/jovyan/workspace).
- Open and execute the notebook. The interactive script will prompt for the following operational inputs:
 - User initials (for tracking provenance)
 - Temporal boundary constraints (Format: YYYYMM start and end dates)
 - Vertical boundary depth constraints (Minimum and maximum depth values in meters)

⚠ The default time and depth range are **2010-01-01T00:00:00-2010-01-31T23:59:59** and **0-10m** for the **global ocean**, bear in mind that this is also the default values in the jupyter notebook `01_EWB_BEACON=merged-v1.5.4_FILTER=BDI-FeatureType_VAR=allEOV_OUT=parquet.ipynb` in step 1, thus **if you change the default values you will need to change you notebook as well.**

⚠ Step 0 must be executed completely before running any other step in the pipeline. It automatically provisions the standardized directory tree structure ([section 7.1.1](#)) necessary to run the workflow.

7.2.1.2. Step 1 - Beacon query

1. Within JupyterLab, navigate into the freshly provisioned path:
`1_DataLakeQuery/Merged_notebooks/`
2. Open and run the primary extraction notebook: `01_EWB_BEACON=merged-v1.5.4_FILTER=BDI-FeatureType_VAR=allEOV_OUT=parquet.ipynb`
3. Provide your active Blue-Cloud authorization token when prompted by the script.

Output: A unified, harmonized dataset compiled in Apache Parquet format, filtered by BDI and feature type, deposited in `1_DataLakeQuery/outputs/`.

⚠ Authentication Note: Tokens must be retrieved via the top panel: How-to → Authorization-How-to as described in [section 5.2.2](#). Tokens expire after 24 hours. If an error occurs during execution, generate a new token, apply it to the environment, and restart the JupyterLab server.

7.2.1.3. Step 2 - CWdt

1. Access the CCP tab from the main VLab dashboard.
2. Select and launch the CWdt execution method (detailed in [section 6.2](#)).
3. Populate the required parameter fields:
 - i. Direct the tool's input parameter to the Parquet data files generated in [Step 1](#).
 - ii. Mount the configuration file (`cw_user_config.yaml`) created during [Step 0](#) and the rest of information needed.
4. Execute the CCP method.

Output: A structured output located at `2_CWduplicates-tool/outputs/` containing explicit, indexed Duplicate List, Discard List and logs in .csv format.

7.2.1.4. Step 3 - FF

1. From the CCP dashboard, launch the FileForge method instance.
2. Populate the required parameter fields (detailed in [section 6.3.4](#))
 - i. Point to the raw outputs from Step 2.

- ii. Select the appropriate operational template file located in `3_FileForge/Configuration_files/` (e.g., `Wb2__config.yaml`).
3. Execute the method to clean or flag overlapping profiles based on the selected criteria.

Output: The final compiled files are pushed back to the user workspace environment under a dedicated execution path, such as:

`3_FileForge/outputs/output/wb2_merged.txt`.

7.2.1.5. *Step 4 - webODV*

1. Copy the final .txt dataset generated by FileForge in Step 3 into the dedicated webODV synchronization folder: `/home/jovyan/4_webODV/`
2. Open the webODV tab from the EWB interface
3. Initialize an import routine to convert the text spreadsheet matrices into an active, compressed .odv collection.

Output: A fully validated, interactive .odv data collection ready for research dissemination or localized export.

[Video tutorial of the full EWB end user workflow](#) can be found at the following path in the EWB workspace: `/workspace/VREFolders/Eutrophication-Workbench/0_Documentation/EWB_demo.mp4`

8. Recommendations for BDIs

During the development and cross-infrastructure integration phases of the EWB workflow, several systematic data disharmonies, metadata gaps, and structural inconsistencies were identified across the contributing BDIs.

To realize true machine-to-machine (M2M) interoperability and support automated cloud harvesting engines like beacon, the EWB framework establishes a formalized data feedback loop. This section details concrete, actionable recommendations directed at the primary data providers to improve the semantic and structural quality of upstream marine data products.

8.1. Cross-BDI Structural Recommendations

8.1.1. Publication and governance of machine-readable vocabularies

To facilitate smooth semantic mapping layers—both manual and automated machine-to-machine integrations—all BDIs must publish their internal code lists, platform designations, and data

categorization schemas using open, web-accessible controlled vocabulary servers (e.g., NERC Vocabulary Server, SeaDataNet Vocabularies).

The Challenge: Systems frequently rely on hardcoded text strings or obscure, closed internal code tables (such as Copernicus Marine Service data type Bigrams or World Ocean Database internal code lists).

The Recommendation: Migrate all proprietary code tables into standardized, machine-resolvable URI formats. This allows semantic mediation layers to translate incoming records on the fly without breaking pipeline parsing scripts.

8.1.2. Enrichment of 3 essential metadata : Cruise identifier, Platform and Instrument codes

Only well-described data can be properly reused by users. In the EWB Vlab use case, which relies in part on data collected during oceanographic cruises, the identification of duplicate records is effective only if three additional key metadata elements, besides date and position, are provided: the cruise identifier, the platform code, and the measuring instrument code.

The Challenge: Depending on the BDI, one of the 2 following metadata is typically used in preference : the cruise identifier or the platform code. Regarding the instrument code, it is either optional in at least one BDI or does not provide sufficient granularity.

The Recommendation: Extend the metadata schema to include both the cruise identifier and the platform codes, where applicable. Data providers should be encouraged not only to supply instrument codes, but also to do so at the finest level of granularity, i.e. at the level of individual parameters.

8.1.3. Implementation of globally unique granular identifiers

Duplicate matching algorithms require immutable anchors to track data provenance across distributed environments.

Feedbacks to BDIs and to data originators, following data processing through a workflow or a value chain, can be delivered efficiently provided that unique identifiers exist to precisely specify where and to what the feedback applies.

The Challenge: Data aggregates often lose granularity, assigning identifiers at the parent file level rather than tracking individual physical samples.

The Recommendation: Implement and enforce a globally unique identifier per profile, per time series, and per trajectory. These granular IDs must persist through all data extraction, aggregation, and reprocessing lifecycles.

8.2. BDI-Specific Targeted Recommendations

8.2.1. EMODnet Chemistry: Adopting standard feature types

To align fully with modern data lake discovery paradigms, EMODnet Chemistry must natively implement and expose explicit structural classification metadata within its core delivery models.

Action Item: Formally integrate the standard Climate and Forecast (CF) Conventions for FeatureTypes into the primary data schemas.

Reference Specification: Geometries must be strictly validated against the official NERC Vocabulary Server collection N03: CF Convention Feature Types ([NVS Collection N03](#)).

By explicitly tagging collections as `profile`, `timeSeries`, or `trajectory`, downstream tools can route datasets to appropriate analytical modules without inspecting every coordinate array.

8.2.2. Copernicus Marine Service (CMEMS): Disentangling feature geometries and QC protocols

Data audits via the beacon engine revealed a series of geometry misalignments and unresolvable vertical references within CMEMS Biogeochemical (BGC) products.

Geometry Misalignments: Under the `feature_type` property, hybrid values (such as `trajectoryProfile` or `timeSeriesProfile`) are sometimes grouped inappropriately with legacy platform identification bigrams. For example the query below shows all the BIGRAMS associated when querying only `'trajectoryProfile'`, `'timeSeriesProfile'`:

```
# Observed structural disharmony in CMEMS active metadata arrays:
Unique_values_for_FEATURE_TYPE = ['trajectoryProfile', 'timeSeriesProfile']
Unique_values_for_BIGRAM       = ['GL', 'XB', 'BO', 'CT', 'SM', 'XX', 'ML', 'PF', 'MO']
```

Vertical Reference Anomalies (Missing Depth Fields): The pipeline flagged instances of observation data completely lacking explicit vertical coordinate (depth) references. While these values are labeled with a Quality Control flag of QC=9 (missing value), providing data records completely decoupled from their physical vertical context limits data utility. CMEMS should establish an automated validation feedback loop back to the originating data networks to guarantee that no physical profiles are published without localized spatial boundaries.

8.2.3. World Ocean Database (WOD): Harmonizing documentation and controlled terms

The World Ocean Database contains significant data volumes, but suffers from text string discrepancies within its metadata headers that complicate automated semantic parsing.

The Challenge: The text labels populated within the internal dataset metadata attribute frequently contradict the definitions and terms officially published in the WOD User Guide, companion web portals, and formal reference publications.

The Recommendation: Standardize the dataset text fields at the source level using invariant, restricted vocabulary lists to eliminate naming variation errors.

8.3. WOD Controlled Vocabulary Standardization Matrix

The table below outlines real-world structural discrepancies identified within the WOD data streams during EWB workflow integration, alongside the proposed controlled vocabularies needed to secure seamless machine interoperability:

Attribute Context	Observed text variance in data files	Reference source / discrepancy link	Proposed Controlled Vocabulary Constraint
Dataset Code	OSD, CTD, XBT, MBT	Inconsistent abbreviations vs. expanded terminology in documentation text.	Map strictly to standard instrumentation definitions (e.g., SeaDataNet L05 platform categories).
Originating Institution	Varied alphanumeric strings (e.g., NODC-USA, NCEI, US-NODC)	NCEI WOD Select Definitions	Enforce standardized EDMO (European Directory of Marine Organisations) codes or global SeaDataNet institution registers.
Platform / Ship Identification	Mixed numerical indexes and unvalidated free-text strings.	Discrepancies between official WOD cruise publications and embedded file headers.	Restrict entry inputs to standardized ICES platform codes or WMO numbers.

Table 6 – WOD Vocabulary standardization matrix

“Dataset” label in downloaded file (and held in Beacon’s monolith)	Comments	Recommendation
“CTD”	Matches CTD in the official WOD codelist	adopt existing or create controlled vocab URIs

“glider”	Matches GLD in the official WOD codelist	adopt existing or create controlled vocab URIs
“drifting buoy”	Matches DB in the official WOD codelist	adopt existing LO6 or create controlled vocab URIs
“profiling float”	Matches PFL in the official WOD codelist	adopt existing or create controlled vocab URIs
“STD”	Not in the official WOD codelist. According to the WOD23 User manual, the “STD” “probe type” can be in a “OSD” dataset if they are low resolution or in a “CTD” dataset if they are high resolution.	adopt existing or create controlled vocab URIs
“bottle/rosette/net”	Not in the official WOD codelist. The closest would be Ocean Station Data (“OSD”) i.e. low resolution profile data from a range of collection and instrument types	
“towed CTD”	Matches UOR in the official WOD codelist	
“moored buoy”	Matches MRB in the official WOD codelist	

Table 7 – Examples of text found in WOD that would need to be constrained using controlled vocabularies to facilitate interoperability.

8.4. Conclusion: Closing the Feedback Loop

Building an interoperable marine data space is an iterative process. By establishing this programmatic feedback framework, the Eutrophication Workbench provides data providers with empirical evidence of data anomalies directly encountered during high-density workflows. BDIs are strongly encouraged to adopt these structural recommendations to move away from legacy, ad-hoc data structures and move toward fully automated, cloud-native FAIR data streams.

9. References

Martín Míguez, B., Heslop, E., Bax, N., Benedetti-Cecchi, L., Canonico, G., Currie, K.I., Evans, K., Fischer, A.S., Garçon, V., Hood, M., Karstensen, J., Lara-López, A., Legler, D.M., Muller-Karger, F.E.E., Nair Thayannur Mullachery, B., Nordlund, L.M.M., Palacz, A.P.P., Post, J., Simmons, S.E., Speich, S., Stukonytė, L., Sutton, A.J.J., Tanhua, T., Telszewski, M., von Schuckmann, K., Waite,

A.M.M., Yu, W., 2026. GOOS Essential Ocean Variables: the backbone of a sustained and evolving global ocean observing system. Front. Mar. Sci. 13. <https://doi.org/10.3389/fmars.2026.1737002>

Reyes Suarez, N.C., 2026a. Jupyter Notebook - Querying EMODnet Chemistry Data via BEACON Monolithic Instance. <https://doi.org/10.5281/zenodo.20036495>

Reyes Suarez, N.C., 2026b. Jupyter Notebook - Eutrophication Workbench Master Setup User Workflow. <https://doi.org/10.5281/zenodo.19923943>

10. Appendix

10.1. Global harmonized and standardized EOv dataset

Statistics from the above mentioned dataset before deduplication. The files is available at https://data.d4science.org/ctlg/Eutrophication-Workbench/blue-cloud2026_global_aggregated_and_harmonized_eutrophication_dataset

1. Global beacon query of all parameter EOvs per volume.

```
TIME_START = "1921-01-01T00:00:00"
TIME_END = "2024-12-31T23:59:59"
DEPTH_MIN = 0
DEPTH_MAX = 10000
LAT_MIN = -90
LAT_MAX = 90
LON_MIN = -180
LON_MAX = 180
```

```
query = merged_client.query() # Create a new query builder instance
query.add_select_column("COMMON_TIME", alias="TIME")
query.add_select_column("COMMON_LATITUDE", alias="LATITUDE")
query.add_select_column("COMMON_LONGITUDE", alias="LONGITUDE")

#DEPTH
query.add_select_column("COMMON_DEPTH", alias="DEPTH")
query.add_select_column("COMMON_DEPTH_QC", alias="DEPTH_QC")
query.add_select_column("COMMON_DEPTH_UNITS", alias="DEPTH_UNITS")
query.add_select_column("COMMON_DEPTH_P01", alias="DEPTH_P01")
query.add_select_column("COMMON_DEPTH_P06", alias="DEPTH_P06")

# CHLOROPHYLL
query.add_select_column(f"COMMON_{chl_col}", alias= chl_col)
query.add_select_column(f"COMMON_{chl_col}_QC", alias=chl_col + "_QC")
```

```

query.add_select_column(f"COMMON_{chl_col}_UNITS", alias=chl_col + "_UNITS")
query.add_select_column(f"COMMON_{chl_col}_P01", alias=chl_col + "_P01")
query.add_select_column(f"COMMON_{chl_col}_P06", alias=chl_col + "_P06")
query.add_select_column("COMMON_CHLOROPHYLL_L05", alias="CHLOROPHYLL_L05")
query.add_select_column("COMMON_CHLOROPHYLL_L22", alias="CHLOROPHYLL_L22")

#OXYGEN PER VOLUME
query.add_select_column(f"COMMON_OXYGEN_{unit}", alias= oxy_col)
query.add_select_column(f"COMMON_OXYGEN_{unit}_QC", alias=oxy_col + "_QC")
query.add_select_column(f"COMMON_OXYGEN_{unit}_UNITS", alias=oxy_col + "_UNITS")
query.add_select_column(f"COMMON_OXYGEN_{unit}_P01", alias=oxy_col + "_P01")
query.add_select_column(f"COMMON_OXYGEN_{unit}_P06", alias=oxy_col + "_P06")
query.add_select_column("COMMON_OXYGEN_L05", alias="OXYGEN_L05")
query.add_select_column("COMMON_OXYGEN_L22", alias="OXYGEN_L22")

# NITRATE PER VOLUME
query.add_select_column(f"COMMON_NITRATE_{unit}", alias= nit_col)
query.add_select_column(f"COMMON_NITRATE_{unit}_QC", alias=nit_col + "_QC")
query.add_select_column(f"COMMON_NITRATE_{unit}_UNITS", alias=nit_col + "_UNITS")
query.add_select_column(f"COMMON_NITRATE_{unit}_P01", alias=nit_col + "_P01")
query.add_select_column(f"COMMON_NITRATE_{unit}_P06", alias=nit_col + "_P06")
query.add_select_column("COMMON_NITRATE_L05", alias="NITRATE_L05")
query.add_select_column("COMMON_NITRATE_L22", alias="NITRATE_L22")

# NITRATE PLUS NITRITE PER VOLUME
query.add_select_column(f"COMMON_NITRATE_NITRITE_{unit}", alias=nit_nit_col)
query.add_select_column(f"COMMON_NITRATE_NITRITE_{unit}_QC", alias=nit_nit_col + "_QC")
query.add_select_column(f"COMMON_NITRATE_NITRITE_{unit}_UNITS", alias=nit_nit_col +
"_UNITS")
query.add_select_column(f"COMMON_NITRATE_NITRITE_{unit}_P01", alias=nit_nit_col + "_P01")
query.add_select_column(f"COMMON_NITRATE_NITRITE_{unit}_P06", alias=nit_nit_col + "_P06")
query.add_select_column("COMMON_NITRATE_NITRITE_L05", alias="NITRATE_NITRITE_L05")
query.add_select_column("COMMON_NITRATE_NITRITE_L22", alias="NITRATE_NITRITE_L22")

# AMMONIUM PER VOLUME
query.add_select_column(f"COMMON_AMMONIUM_{unit}", alias= amm_col)
query.add_select_column(f"COMMON_AMMONIUM_{unit}_QC", alias=amm_col + "_QC")
query.add_select_column(f"COMMON_AMMONIUM_{unit}_UNITS", alias=amm_col + "_UNITS")
query.add_select_column(f"COMMON_AMMONIUM_{unit}_P01", alias=amm_col + "_P01")
query.add_select_column(f"COMMON_AMMONIUM_{unit}_P06", alias=amm_col + "_P06")
query.add_select_column("COMMON_AMMONIUM_L05", alias="AMMONIUM_L05")
query.add_select_column("COMMON_AMMONIUM_L22", alias="AMMONIUM_L22")

# PHOSPHATE PER VOLUME
query.add_select_column(f"COMMON_PHOSPHATE_{unit}", alias= pho_col)
query.add_select_column(f"COMMON_PHOSPHATE_{unit}_QC", alias=pho_col + "_QC")

```



```

query.add_select_column(f"COMMON_PHOSPHATE_{unit}_UNITS", alias=pho_col + "_UNITS")
query.add_select_column(f"COMMON_PHOSPHATE_{unit}_P01", alias=pho_col + "_P01")
query.add_select_column(f"COMMON_PHOSPHATE_{unit}_P06", alias=pho_col + "_P06")
query.add_select_column("COMMON_PHOSPHATE_L05", alias="PHOSPHATE_L05")
query.add_select_column("COMMON_PHOSPHATE_L22", alias="PHOSPHATE_L22")

# SILICATE PER VOLUME
query.add_select_column(f"COMMON_SILICATE_{unit}", alias=sil_col)
query.add_select_column(f"COMMON_SILICATE_{unit}_QC", alias=sil_col + "_QC")
query.add_select_column(f"COMMON_SILICATE_{unit}_UNITS", alias=sil_col + "_UNITS")
query.add_select_column(f"COMMON_SILICATE_{unit}_P01", alias=sil_col + "_P01")
query.add_select_column(f"COMMON_SILICATE_{unit}_P06", alias=sil_col + "_P06")
query.add_select_column("COMMON_SILICATE_L05", alias="SILICATE_L05")
query.add_select_column("COMMON_SILICATE_L22", alias="SILICATE_L22")

# SALINITY
query.add_select_column("COMMON_SALINITY", alias="SALINITY")
query.add_select_column("COMMON_SALINITY_QC", alias="SALINITY_QC")
query.add_select_column("COMMON_SALINITY_UNITS", alias="SALINITY_UNITS")
query.add_select_column("COMMON_SALINITY_P01", alias="SALINITY_P01")
query.add_select_column("COMMON_SALINITY_P06", alias="SALINITY_P06")
query.add_select_column("COMMON_SALINITY_L05", alias="SALINITY_L05")
query.add_select_column("COMMON_SALINITY_L22", alias="SALINITY_L22")

# TEMPERATURE
query.add_select_column("COMMON_TEMPERATURE", alias="TEMPERATURE")
query.add_select_column("COMMON_TEMPERATURE_QC", alias="TEMPERATURE_QC")
query.add_select_column("COMMON_TEMPERATURE_UNITS", alias="TEMPERATURE_UNITS")
query.add_select_column("COMMON_TEMPERATURE_P01", alias="TEMPERATURE_P01")
query.add_select_column("COMMON_TEMPERATURE_P06", alias="TEMPERATURE_P06")
query.add_select_column("COMMON_TEMPERATURE_L05", alias="TEMPERATURE_L05")
query.add_select_column("COMMON_TEMPERATURE_L22", alias="TEMPERATURE_L22")

# add metadata columns
query.add_select_column("COMMON_PLATFORM_L06", alias="PLATFORM_L06")
query.add_select_column("COMMON_PLATFORM_C17", alias="PLATFORM_C17")
query.add_select_column("SOURCE_BDI")
query.add_select_column("SOURCE_BDI_DATASET_ID")
query.add_select_column("COMMON_EDMO_CODE", alias="EDMO_CODE")
query.add_select_column("COMMON_CSR", alias="CSR")

query.add_select_column("COMMON_HARMONIZATION_DATE")
query.add_select_column("COMMON_BDI_SNAPSHOT_DATE")
query.add_select_column("COMMON_BDI_MONOLITH_VERSION")

#metadata from monoliths
query.add_select_column("bigram", alias="BIGRAM")

```

```

query.add_select_column("dataset", alias="WOD_DATASET")
query.add_select_column("COMMON_FEATURE_TYPE", alias="FEATURE_TYPE")
# important to generate the odv format
query.add_select_column("COMMON_ODV_TAG", alias="ODV_TAG")

# Apply filters to the query. WE KEEP ONLY SAMPLES WITH ASSOCIATED TEMPERATURE AND SALINITY
# MEASUREMENTS
query.add_filter(
    OrFilter([IsNotNullFilter(chl_col),
              IsNotNullFilter(oxy_col),
              IsNotNullFilter(nit_col),
              IsNotNullFilter(nit_nit_col),
              IsNotNullFilter(amm_col),
              IsNotNullFilter(pho_col),
              IsNotNullFilter(sil_col),
              ])
)

query.add_range_filter("TIME", TIME_START, TIME_END) # You can adjust the date range as
# needed. The format is ISO 8601.

query.add_range_filter("DEPTH", DEPTH_MIN, DEPTH_MAX) # Depth range

query.add_range_filter("LATITUDE", LAT_MIN, LAT_MAX) # Latitude range from -90 to 90 for
# full range (you can adjust as needed)
query.add_range_filter("LONGITUDE", LON_MIN, LON_MAX) # Longitude range from -180 to 180 for
# full range (you can adjust as needed)

```

Exploring the global harmonized and standardized eutrophication data collection the following statistics were retrieved:

- Total number of BDI counts: 1316427551
- Data availability per BDI:
-

SOURCE_BDI	COUNT	PERCENTAGE, %
BEACON_WOD	637936353	48.5
BEACON_CMEMS_BGC	625694279	47.5

BEACON_EMODNET_CHEMISTRY

52796919

4.0

Table 8 – Data availability per BDI from the Global Harmonized and Standardized eutrophication data collection.

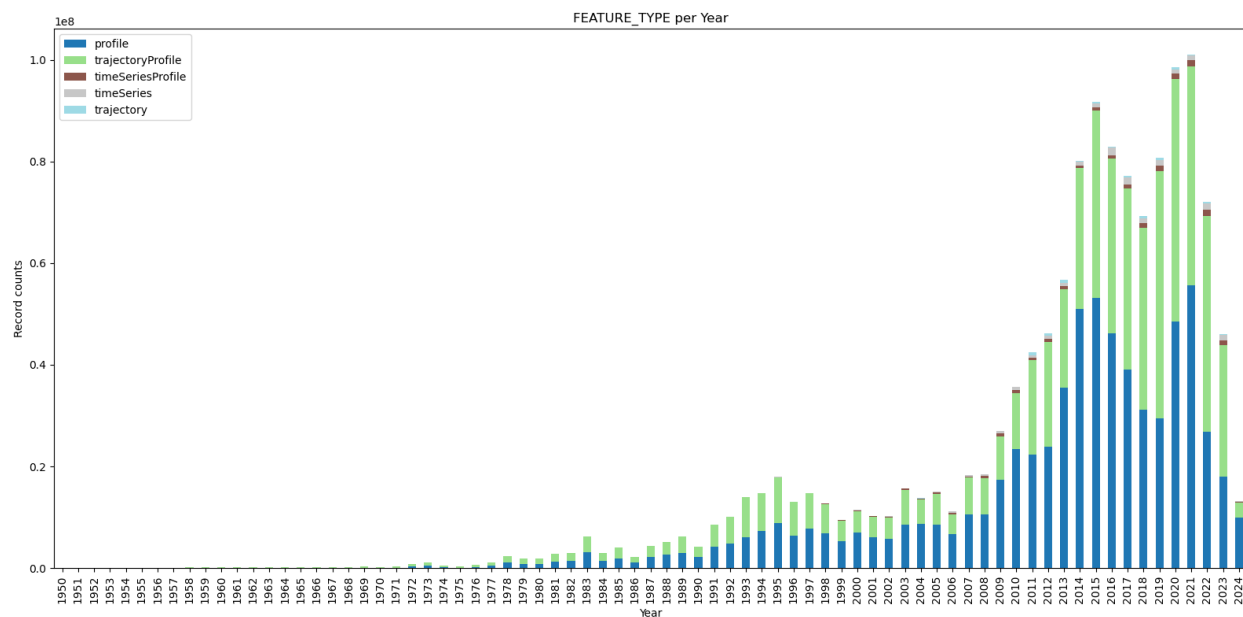


Figure 10 – Record counts and time availability of feature types from the Global Harmonized and Standardized eutrophication data collection.

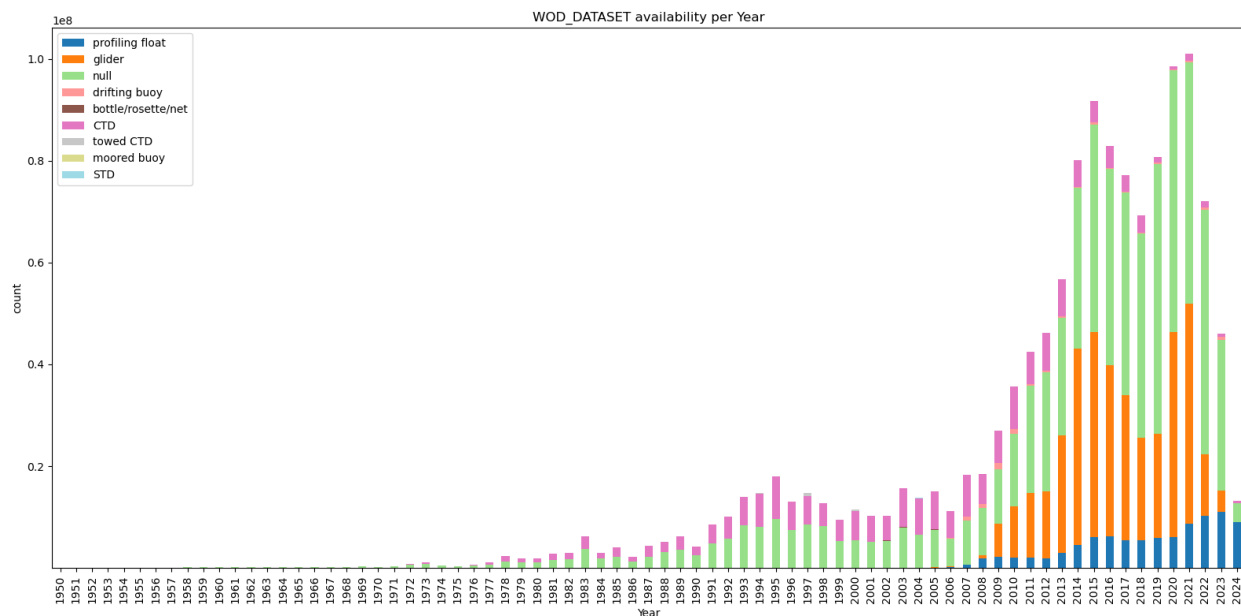


Figure 11 – Record counts and time availability of WOD dataset type from the Global Harmonized and Standardized eutrophication data collection.

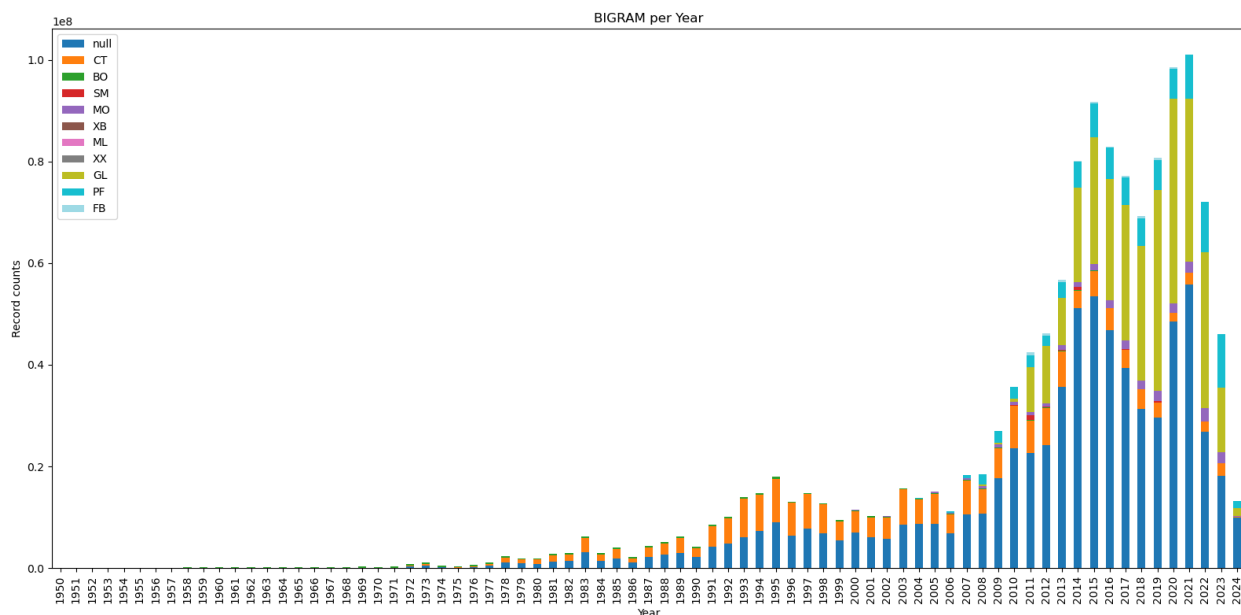


Figure 12 – Record counts and time availability of CMEMS bigrams from the Global Harmonized and Standardized eutrophication data collection.

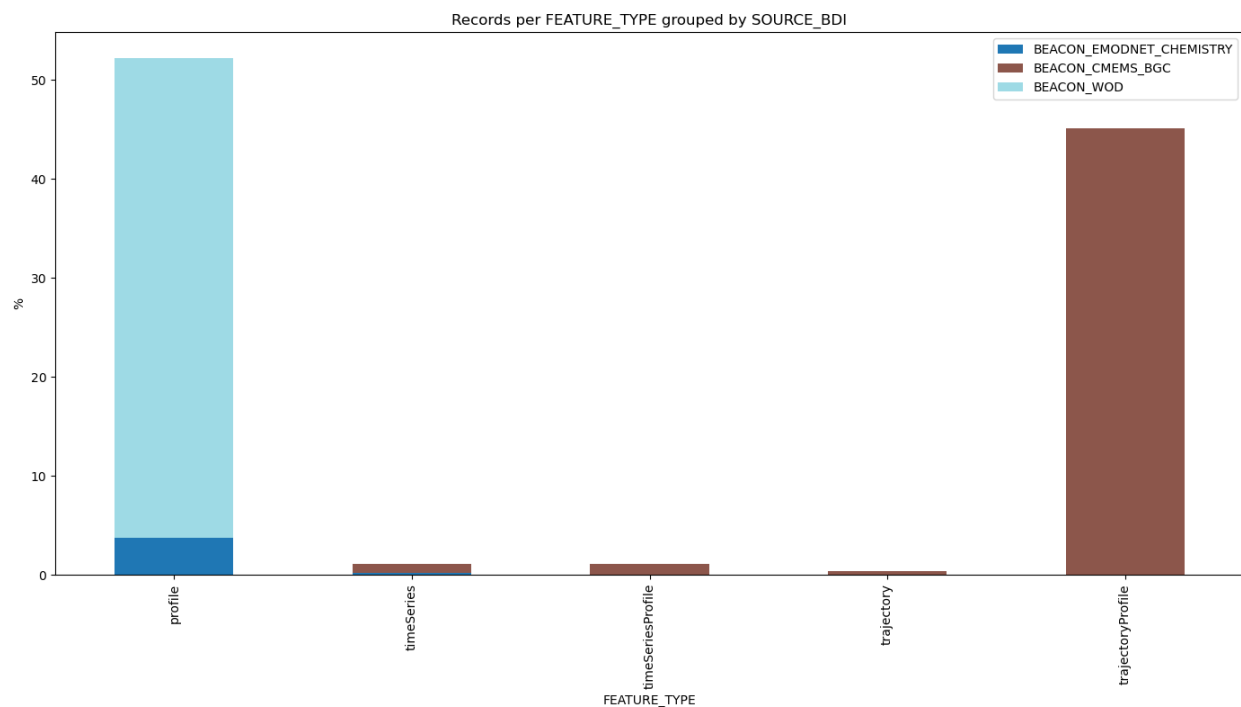


Figure 13 – Record counts vs feature type on each BDI from the Global Harmonized and Standardized eutrophication data collection.

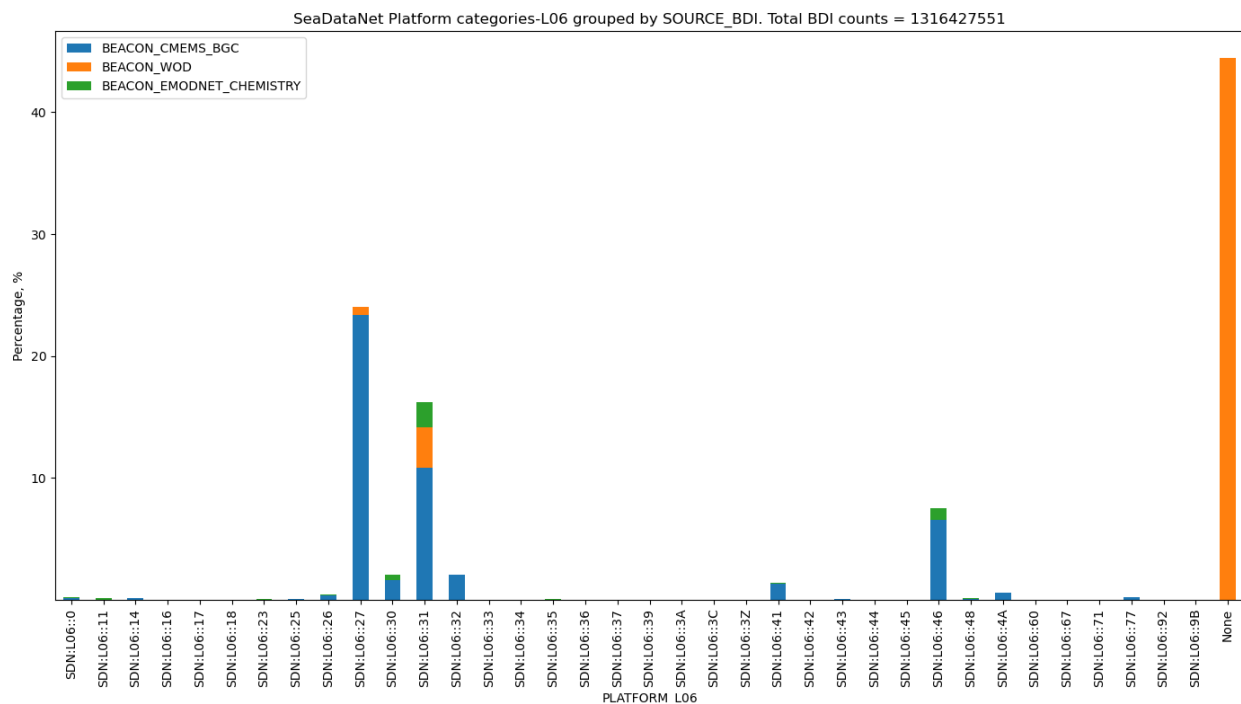


Figure 14 – Percentage of SeaDataNet Platform categories L06 per SOURCE_BDI from the Global Harmonized and Standardized EOVS data collection.